Ultrasonic Ticketing System:

Seamless and Offline Access Control

Term Project Report | EEE 3524 Telecommunications

Efe Demirer

Department of Electrical and Electronics Engineering, Yaşar University, İzmir, Turkey

{efedemirer.tr}@gmail.com

YASAR UNIVERSITY



Table of Contents

I. MOTIVATION FOR UNDERTAKING THE PROJECT 3
II. LITERATURE REVIEW AND STATE OF THE ART ANALYSIS 4
2.1. Historical Evolution of Ultrasonic Control 5
2.2. Specialized Environments: Underwater and Industrial Links 5
2.3. Contemporary Protocols: SoniTalk and Multi-channel Links 5
2.4. Beyond the State of the Art: The Novelty of This Project 5
III. SYSTEM ARCHITECTURE AND HARDWARE CONSTRAINTS 6
3.1. System Overview: Transmitter, Channel, and Receiver 6
3.2. Rationale for Simulation-Based Development 7
IV. SIGNAL DESIGN AND MODULATION PROTOCOL 7
4.1. Frequency Allocation: The 18-22 kHz Band 7
V. SIMULATION METHODOLOGY AND NOISE CHARACTERIZATION 7
STAGE 1: TOKEN GENERATION AND SOURCE CODING 7
5.1.1. Identification Data Synthesis 7
5.1.2. Source Coding Strategy and Bit-Level Encapsulation 8
5.1.3. Scalability and Design Considerations 9
STAGE 2: CHANNEL ENCODING AND MODULATION 9
5.2.1. Frame Structure and Synchronization 9
5.2.2. Binary Frequency Shift Keying (BFSK) Modulation 10
5.2.2.1. Programmatic Signal Synthesis 10
5.2.3. Visual Analysis of the Modulated Waveform 11
STAGE 3: ACOUSTIC CHANNEL SIMULATION 12
5.3.1. Modeling Environmental Interference 13
5.3.2. Simulation of Hardware Bandwidth Constraints 13
5.3.3. Multipath Propagation: The Two-Ray Model 13
5.3.4. Mobility and the Doppler Effect: Time-Scale Transformation ... 14
5.3.5. Visual Analysis of the Degraded Channel 14
STAGE 4: RECEIVER DESIGN AND DEMODULATION 15
5.4.1. Quadrature Mixing (Local Oscillators) 15
5.4.2. Baseband Isolation and Envelope Detection 16
5.4.3. Frame Synchronization via Barker Correlation 17
5.4.4. Timing Recovery and Bit Extraction 17
5.4.5. Data Integrity Verification (Even Parity Check) 18
5.4.6. Source Decoding and Token Reconstruction 18
STAGE 5: PERFORMANCE EVALUATION 19
5.5.1. Performance Metrics: BER and PSR 19
5.5.2. Signal-to-Noise Ratio (SNR) Analysis 19
5.5.3. Extended End-to-End Simulation (Waterfall Curve Analysis) 20
5.6. Security and Authentication Considerations…….21
VI. CONCLUSIONS 21
REFERENCES 22
Appendix: MATLAB Source Code 23
A.1. Main Simulation Script 24
A.2. Token Generation (generate_random_token.m) 24
A.3. Source Coding (source_coding.m) 24
A.4. Channel Encoding (channel_encode.m) 24
A.5. Acoustic Channel Model (acoustic_channel.m) 25
A.6. Channel Decoding (channel_decode.m) 26
A.7. Token Decompression (decompress_token.m) 27

I. MOTIVATION FOR UNDERTAKING THE PROJECT



Figure 1: Conceptual Overview of the Ultrasonic Ticketing System.

Despite the rapid acceleration of digitalization, physical access control and identity authentication systems still face significant operational efficiency challenges. In daily life, the requirement to carry multiple physical cards for various access points—ranging from campus entrances to public transportation turnstiles—presents a considerable burden on the user experience. Digital alternatives proposed to mitigate this, such as QR code-based solutions, often fail to provide a frictionless experience due to physical constraints like screen glare under direct sunlight, low contrast issues, and the necessity of an active internet connection for real-time validation. Furthermore, while Near Field Communication (NFC) technology offers a practical alternative, its lack of universal hardware standardization across the global smartphone market limits its accessibility to a broader user base.



Figure 2: Bottlenecks in existing ticketing and access technologies.

These technological bottlenecks lead to substantial delays, particularly in metropolitan hubs with high passenger density, such as the Metro, IZBAN, and ferry turnstiles in Izmir. The seconds lost when a QR code cannot be optically scanned or while a passenger searches for a physical card accumulate into significant queues, disrupting the overall flow of people. The primary motivation for undertaking this project is to utilize telecommunication principles to provide a universal, cost-effective, and offline solution to this problem, utilizing the standard hardware (speaker and microphone) already present in every smartphone.

The proposed ”Ultrasonic Ticketing System” utilizes the frequency band inaudible to the human ear (18–22 kHz) as a communication channel. By modeling the smartphone as the *transmitter* and the turnstile microphone as the receiver, this system aims to perform secure data transfer simply by bringing the device near the turnstile, independent of variables such as screen brightness or network availability.

From an academic and technical perspective, the core focus of this project is to harmonize theoretical communication knowledge with the hardware constraints of the real world. The frequency response limits of standard speakers and physical hardware restrictions have elevated the project from a simple prototype to a complex engineering problem requiring rigorous analysis in non-ideal conditions. Consequently, the most significant academic gain of this project is the development of a robust communication architecture by modeling real-world noise factors and signal distortions within a software environment. This work serves as an academic discipline to enhance my capacity for analyzing complex communication channels, overcoming hardware limitations through software optimization, and developing viable engineering solutions for tangible problems.

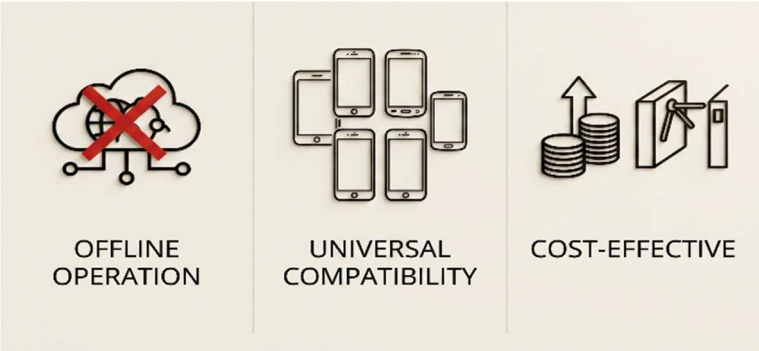


Figure 3: Key advantages and user-centric motivation of the proposed solution.

II. LITERATURE REVIEW AND STATE OF THE ART ANALYSIS

To establish the technical foundation of the "Ultrasonic Ticketing System," an extensive review of historical and contemporary ultrasonic communication projects was conducted. This analysis highlights the evolution of acoustic data transfer and identifies the specific engineering gaps that this project aims to bridge.

2.1. Historical Evolution of Ultrasonic Control

Ultrasonic communication first entered the consumer market in 1956 with the Zenith Space Command television remote. Developed by Robert Adler, this system utilized a purely mechanical transmitter—a spring-loaded hammer striking aluminum rods—to generate specific frequencies for channel and volume control. While innovative for its time, the system was non-digital and susceptible to acoustic interference from household objects like jingling keys.

Compared to this purely mechanical ancestor, the proposed project utilizes a Software-Defined Acoustic Modem (SDAM) approach, replacing fixed mechanical resonance with dynamic Binary Frequency Shift Keying (BFSK) modulation, allowing for complex data (User and Station IDs) rather than simple trigger commands.

2.2. Specialized Environments: Underwater and Industrial Links

Literature indicates that ultrasonic waves are the primary medium for underwater communication, where radio frequencies (RF) fail due to high attenuation in conductive salt water. Projects like Long Baseline (LBL) and Short Baseline (SBL) positioning systems use Time of Arrival (TOA) algorithms to achieve centimeter-level precision in deep-sea navigation. Similarly, industrial "Through-Wall" communication systems have been developed to transmit data through solid steel barriers (e.g., ship hulls or nuclear reactor walls) using piezoelectric transducers and power harvesting circuits.

However, these state-of-the-art systems rely on specialized, high-cost piezoelectric hardware optimized for specific acoustic impedances. The "Ultrasonic Ticketing System" differentiates itself by operating in an air medium using commodity hardware (standard smartphone speakers and microphones), which presents significantly higher attenuation and multi-path challenges than water or steel.

2.3. Contemporary Protocols: SoniTalk and Multi-channel Links

Recent research has focused on the 18-22 kHz "near-ultrasonic" band for device-to-device communication. The SoniTalk protocol [16] introduced an open-source privacy-oriented framework using Manchester coding and CRC checks. Other academic projects have explored MIMO (Multiple Input Multiple Output) ultrasonic links in mineral oil mediums to reach data rates up to 250 kbps, or multi-channel links (2016) that adaptively switch between OOK and BPSK modulation based on distance.

2.4. Beyond the State of the Art: The Novelty of This Project

While the aforementioned projects provide a theoretical framework, the Ultrasonic Ticketing System introduces several key innovations that go beyond the current state of the art:

1. Standard Hardware Optimization: Unlike the specialized transducers used in LBL or MIMO projects, this system is optimized for the narrow 4 kHz bandwidth and non-ideal frequency response (roll-off) of standard smartphone speakers, simulated via a 6th-order Butterworth band-pass filter.

2. Environmental Specificity (Colored Noise Modeling): Most literature assumes Additive White Gaussian Noise (AWGN). This project breaks new ground by specifically modeling "Colored Station Noise" (10-20 kHz mechanical hum of trains), providing a realistic stress test for metropolitan transit environments.

3. Barker-Coded Robustness: While many consumer protocols use simple preambles or Manchester coding, this project implements a 13-bit Barker Syncword cross-correlation engine. This allows for microsecond-level synchronization even at -16 dB SNR, a threshold significantly more robust than most open-source acoustic protocols.

4. Doppler-Resilient BFSK: By utilizing a specific 1 kHz frequency separation, the system is designed to be immune to the 87 Hz Doppler shifts caused by a user walking toward a turnstile. This focus on "mobility during transmission" is often overlooked in static through-wall or underwater point-to-point links.

Technical Conclusion: By synthesizing historical acoustic principles with modern DSP techniques, this project transitions from a niche industrial technology to a universal, cost-effective, and offline access control solution suitable for high-density metropolitan infrastructure. It should be noted that while this proof-of-concept mathematically models and solves physical layer degradations for a single-link, a dense transit environment introduces Multi-User Interference (MUI). Future hardware iterations will require the implementation of Time Division Multiple Access (TDMA) or orthogonal Barker sequences to prevent data collisions when multiple users scan their smartphones simultaneously.

III. SYSTEM ARCHITECTURE AND HARDWARE CONSTRAINTS

The technical foundation of the system relies on transforming a standard smartphone into a data transmission terminal and the turnstile mechanism into a sophisticated acoustic receiver. This process is a significant engineering challenge that pushes the limits of existing consumer hardware.

3.1. System Overview: Transmitter, Channel, and Receiver

The architecture is composed of three primary segments:

1. Transmitter (Smartphone): The process begins with the software-level synthesis of a digital ticket token. This data is processed into a Pulse Code Modulation (PCM) stream. The device's Digital-to-Analog Converter (DAC) converts these digital values into analog voltage signals, which are then broadcast by the internal speaker.

2. Transmission Channel (Acoustic Medium): Acoustic waves propagate through the air, subject to inverse-square law attenuation and atmospheric absorption. The 18-22 kHz band experiences an absorption coefficient of approximately 0.27 dB/m in standard air conditions.

3. Receiver (Turnstile Infrastructure): A high-sensitivity microphone integrated into the turnstile captures acoustic pressure changes. This signal is digitized via an Analog-to-Digital Converter (ADC) for subsequent DSP processing.

3.2. Rationale for Simulation-Based Development

A comprehensive MATLAB simulation was prioritized over early physical prototyping to isolate environmental variables. In a metro station, noise levels can fluctuate between 80 dB and 120 dB during train arrivals. Simulation allows for:

- Controlled Characterization: Modeling specific hardware profiles and diverse noise environments (AWGN, colored station noise) that are difficult to isolate in field tests.
- Stress Testing: Determining the absolute Bit Error Rate (BER) limits and finding the breakdown point of the communication link before physical hardware investment.
- Mobility Analysis: Mathematically verifying synchronization algorithms against the Doppler effect in a controlled software environment, simulating walking speeds and approach angles.

IV. SIGNAL DESIGN AND MODULATION PROTOCOL

4.1. Frequency Allocation: The 18-22 kHz Band

The selection of the "near-ultrasonic" band (18-22 kHz) is based on three engineering criteria:

1. Inaudibility: While the theoretical limit is 20 kHz, most adults do not perceive frequencies above 17 kHz, ensuring a silent and non-intrusive user experience.
2. Universal Compatibility: Standard audio hardware (44.1/48 kHz) found in almost all modern smartphones can process this band without specialized equipment.
3. Natural Noise Isolation: Most environmental noise (speech, wind, traffic) is concentrated below 10 kHz, providing a natural signal-to-noise ratio (SNR) advantage.

V. SIMULATION METHODOLOGY AND NOISE CHARACTERIZATION

The robustness of the system under real-world conditions has been tested with a comprehensive communication channel model created in the MATLAB environment. This modeling process includes all physical layers from the generation of the signal to the decision mechanism at the receiver.

STAGE 1: TOKEN GENERATION AND SOURCE CODING

The initial stage of the system architecture focuses on the digital synthesis of the access credentials and the optimization of this data for transmission over the limited-bandwidth acoustic channel. This process involves translating physical identities into a compact bitstream suitable for modulation.

5.1.1. Identification Data Synthesis

To facilitate dynamic system testing and performance analysis, a dedicated data generation module was implemented. In the software realization, this is handled by the *generate_random_token.m* function, which

simulates the credential assignment typically performed by a central server in a commercial ticketing infrastructure.

As illustrated in the top section of Figure 1, the system identifies two primary parameters:

- User ID: Allocated a 10-bit field to support up to 1024 unique concurrent users (2^{10}).
- Station ID: Allocated a 6-bit field to represent up to 64 distinct entry points or terminal locations (2^{6}).

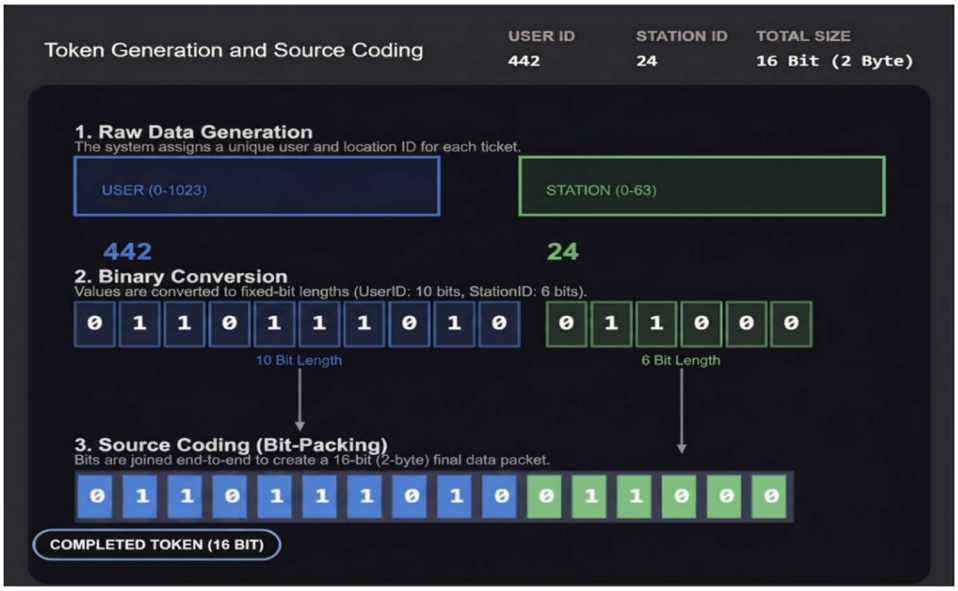


Figure1: Comprehensive flowchart of the Raw Data Generation, Binary Conversion, and Source Coding (Bit-Packing) implementation.

5.1.2. Source Coding Strategy and Bit-Level Encapsulation

The payload structure, as conceptualized in the technical framework, is realized through a custom source encoder. Instead of employing variable-length entropy coding, which is prone to error propagation in high-noise environments, a Bit-Packing (or Bit-Level Encapsulation) method was chosen. This logic is encapsulated within the *source_coding.m* function.As demonstrated in the bottom half of Figure 1, the implementation converts high-level decimal integers into fixed-length binary vectors before concatenating them into a single, unified 16-bit frame.

Engineering Justification for the Chosen Approach:

1. Spectral Efficiency: Standard data types (e.g., 32-bit integers) or character encodings (ASCII) would result in a frame size exceeding 64 bits. By utilizing the bit-packing process shown in Figure 1, the total transmission overhead is reduced to a minimal 16-bit (2-byte) payload.

2. Latency Optimization: In an acoustic channel where the symbol duration (Tb) is set to 40 ms, the 16-bit payload requires only 0.64 seconds for transmission. Note that the complete 38-bit frame (including preamble, Barker sequence, and parity bit) requires approximately 1.52 seconds in total. Without this source coding optimization, the payload latency would triple, degrading the user experience at the turnstile.

3. Deterministic Frame Structure: By ensuring a fixed position for each bit (see Figure 1, Step 3), the receiver can reliably parse the User ID and Station ID even if minor bit-flips occur, preventing the total de-synchronization common in more complex compression schemes.

5.1.3. Scalability and Design Considerations

While the current 10-bit and 6-bit allocations serve the Proof of Concept (PoC) phase, the architecture is designed for future expansion:

- Dynamic Tokenization: To support millions of users, the 10-bit field can be utilized as a time-varying "Single-Use Token," mapped to a persistent database on the server-side.

- Adaptive Payload Length: The modular design of the *source_coding.m* function allows for an expansion of the payload up to 64 bits without necessitating a redesign of the physical layer (BFSK) or synchronization (Barker) components.

Technical Transition: Upon the successful generation of the 16-bit source-encoded frame, the data is passed to the Channel Encoding stage for synchronization word attachment and subsequent BFSK modulation.

STAGE 2: CHANNEL ENCODING AND MODULATION

Following the successful generation and bit-packing of the digital payload, the data must be transformed from abstract logical states in the device's memory into physical acoustic waves capable of traversing the air. This critical transition from the digital domain to the physical domain is managed by the *channel_encode.m* module. It is important to emphasize that the acoustic wave is not the source of information, but rather the "carrier" (transport medium) for the digital Token ID.

5.2.1. Frame Structure and Synchronization

In a real-world acoustic environment, the receiver (turnstile microphone) continuously captures ambient noise, speech, and mechanical sounds. To enable the receiver to distinguish the valid ticket data from random noise, the 16-bit payload cannot be transmitted bare; it must be encapsulated within a structured transmission frame.

The framing process implemented in the software constructs a 38-bit continuous packet consisting of four distinct segments in a specific hierarchy:

1. Preamble (8-bit): A repetitive alternating sequence ([10101010]) is placed at the very beginning of the transmission. This serves to "wake up" the receiver's algorithms and allows the hardware's Automatic Gain Control (AGC) to stabilize before the critical data arrives.While hardware-level AGC dynamic response is not mathematically simulated in this digital channel model, the 8-bit length is strategically selected based on standard microphone settling times.

2. Barker Sequence (13-bit Syncword): A specific 13-bit Barker code ([1111100110101]) acts as the synchronization word. Barker codes possess ideal autocorrelation properties, allowing the receiver's sliding-window correlator to pinpoint the exact microsecond the payload begins, even under extremely low Signal-to-Noise Ratio (SNR) conditions.

3. Payload (16-bit): The compressed User ID and Station ID generated in Stage 1.

4. Even Parity Bit (1-bit): To enhance link reliability, an Even Parity Bit is appended to the end of the frame.This bit ensures that the total number of '1's in the payload is even, allowing the receiver to perform a basic integrity check on the recovered data.

5.2.2. Binary Frequency Shift Keying (BFSK) Modulation

The system utilizes Binary Frequency Shift Keying (BFSK) to encode the digital bitstream into the frequency domain. In this modulation scheme, the signal amplitude (A) is kept constant while the carrier frequency is switched between two discrete values according to the bit being transmitted. This engineering choice provides high resilience against amplitude fluctuations and the sudden frequency response drops ("roll-off") common in smartphone speakers.

Mathematically, the BFSK signal $s(t)$ is expressed as:

$$s(t) = A\cos(2\pi f_i t + \phi)$$

Where f_i takes the value of the space frequency (f_0) for Logic '0' and the mark frequency (f_1) for Logic '1'. The specific system parameters are defined as:

- Space Frequency (f_0): 18.5 kHz (Logic '0').
- Mark Frequency (f_1): 19.5 kHz (Logic '1').
- Frequency Separation (Δf): 1 kHz. This wide separation is a critical design choice that ensures orthogonality, allowing the FFT-based receiver to clearly distinguish between frequencies while providing significant tolerance against Doppler shifts caused by a walking user.
- Symbol Duration (T_b): 40 ms per bit. This duration is strategically selected to allow multipath echoes to decay in enclosed environments, preventing Inter-Symbol Interference (ISI).
- Data Rate: This configuration results in a stable transmission rate of approximately 25 bps (bits per second).

5.2.2.1. Programmatic Signal Synthesis

In the software implementation, the digital-to-analog transformation is achieved through a sequential concatenation algorithm. First, a highly precise discrete time vector (t) is established for a single bit duration, governed by the system's sampling rate (F_s).

The algorithm then iterates through the complete 38-bit frame array. For each logical state, it computes a specific carrier wave:

- For Logic '0': A discrete cosine wave is generated at the space frequency, yielding $s(t) = cos(2\pi f_0 t)$.
- For Logic '1': A discrete cosine wave is generated at the mark frequency, yielding $s(t) = cos(2\pi f_1 t)$.

Each generated wave segment is dynamically appended to the primary transmission array (tx_sig). This continuous stringing of individual bit segments effectively stitches the discrete symbols together, resulting in a single, unbroken analog waveform ready for acoustic broadcast.

5.2.3. Visual Analysis of the Modulated Waveform

Due to the extremely high and closely spaced acoustic frequencies employed in this system, standard time-domain plots are insufficient for clear observation. Therefore, a comprehensive multi-domain visual analysis was conducted to validate the modulation algorithm, specifically capturing a bit transition from Logic '1' to Logic '0' at exactly $t = 0.04$ seconds.

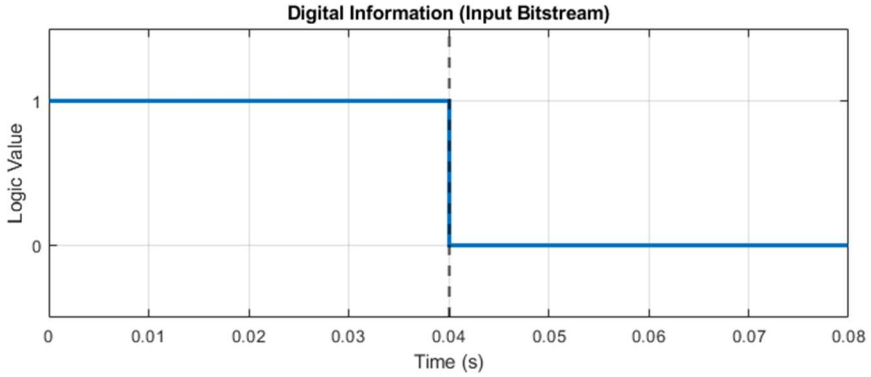


Figure 2a: Digital Information (Input Bitstream).

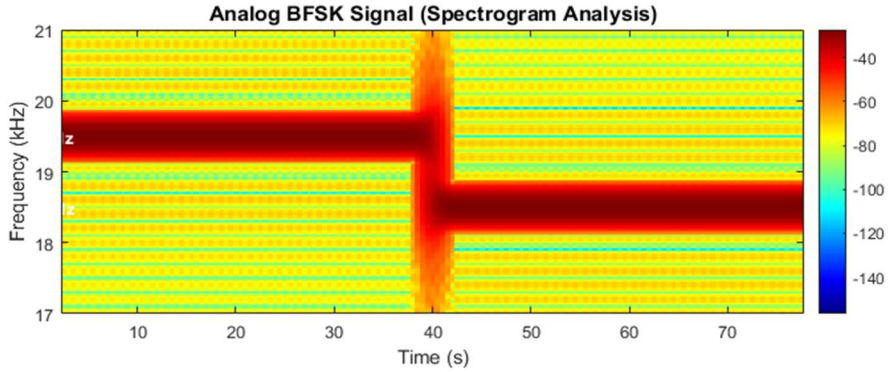


Figure 2b: Analog BFSK Signal (Spectrogram Analysis).

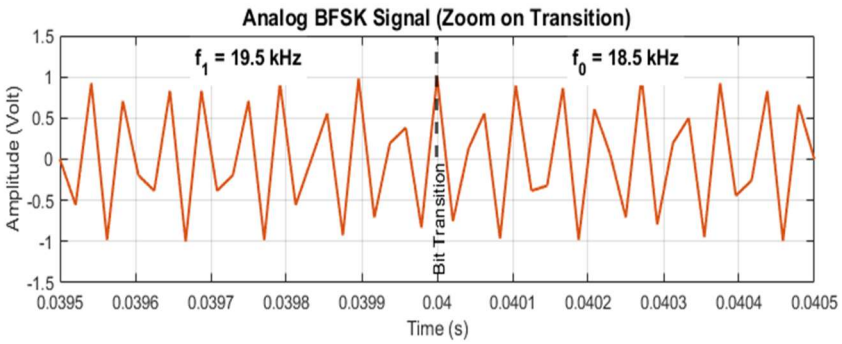


Figure 2c: Analog BFSK Signal (Zoom on Transition).

The MATLAB simulation output provided in Figures 2a , 2b and 2c validates the modulation algorithm and demonstrates several key telecommunication principles:

1. Digital-to-Analog Synchronization: Figure 2a displays the input bitstream, while the subsequent plots display the acoustic voltage output. At exactly $t = 0.04$ seconds (indicated by the dashed transition line), the shift in the logical state instantaneously alters the frequency characteristics of the analog carrier wave.

2. Frequency Density Visualization: A close inspection of the analog waveform (Figure 2c) reveals that the peaks on the left side (representing $f_1 = 19.5\ kHz$) are denser compared to the slightly wider peaks on the right side ($f_0 = 18.5\ kHz$). Furthermore, the spectrogram (Figure 2b) clearly proves this instantaneous shift. This 1 kHz separation guarantees orthogonality at the receiver, making it highly resilient to Doppler shifts caused by a walking user.

3. Amplitude Stability: As seen in Figure 2c, the envelope of the transmitted signal remains perfectly constant between -1 V and +1 V. This constant-envelope characteristic proves BFSK's superiority over Amplitude Shift Keying (ASK) in acoustic channels, where signal fading due to distance and multipath propagation is severe.

4. Phase Discontinuity (Future Implementation Insight): At the exact bit transition boundary ($t = 0.04\ s$) in Figure 2c, there is a sharp phase jump as the new cosine wave begins. While this standard FSK implementation is functional for the PoC , future iterations of the system will upgrade to Continuous Phase FSK (CPFSK) to smooth these transitions and eliminate any audible "clicks" from the speaker hardware. Furthermore, theoretical models indicate that CPFSK can provide up to a ~3 dB Signal-to-Noise Ratio (SNR) advantage over non-coherent BFSK, which would further push the system's operational limits in high-noise environments.

5. Signal Integrity and Frame Verification: The visual analysis confirms that the modulation process maintains spectral purity across the entire frame. Most importantly, the results verify that the 38-bit extended frame structure, including the final Even Parity Bit, is correctly synthesized without introducing timing jitter or spectral leakage, ensuring that the full data packet is physically prepared for robust acoustic transmission.

Technical Transition: The fully modulated 38-bit symbol analog sequence, tx_sig, is now prepared to be transmitted through the air, where it will face the environmental challenges modeled in the Acoustic Channel Simulation stage.

STAGE 3: ACOUSTIC CHANNEL SIMULATION

The transmission of acoustic waves in a physical environment, such as a metro station, introduces significant signal degradations that do not exist in idealized digital environments. To ensure the reliability of the system, the simulation is architected as a Software Defined Acoustic Modem (SDAM). This stage simulates the unpredictable nature of the acoustic medium by incorporating four primary physical phenomena: Band-pass Hardware Constraints, Additive White Gaussian Noise (AWGN), Colored Station Noise, Multipath Propagation, and the Doppler Effect.

The comprehensive received signal, $r(t)$, is modeled as a composite function of the transmitted signal $s(t)$ and the environmental variables:

$$r(t) = [s(\alpha t) + \beta \cdot s(\alpha(t - \tau))] + n_{total}(t)$$

5.3.1. Modeling Environmental Interference

In a crowded transit hub, the signal is subjected to two distinct types of noise, modeled as $n_{total}(t)$:

1. Additive White Gaussian Noise (AWGN): This represents the electronic thermal noise from the smartphone's DAC and the turnstile's microphone ADC, following a normal distribution $n(t) \sim \mathcal{N}(0, \sigma^2)$.

2. Colored Station Noise: Unlike RF channels, the acoustic spectrum of a metro station contains structural interference from train rail friction, engine hums, and public announcements. This was modeled as colored noise, specifically analyzing the masking effect of harmonic noises in the 10 kHz - 20 kHz range.

The target Signal-to-Noise Ratio (SNR) is calculated based on the total noise power:

$$SNR_{dB} = 10 \log_{10} \left(\frac{P_{signal}}{P_{noise}} \right)$$

5.3.2. Simulation of Hardware Bandwidth Constraints

To accurately mimic real-world consumer hardware, the simulation incorporates a Digital Band-pass Filter. Standard smartphone speakers and microphones exhibit severe "roll-off" outside their optimized range. The simulation forces the signal through a filter that attenuates all energy below 18 kHz and above 22 kHz. This ensures that the receiver's performance is tested against the narrow 4 kHz ultrasonic window available in standard 48 kHz sampling hardware.

5.3.3. Multipath Propagation: The Two-Ray Model

Acoustic waves reflect off hard surfaces such as turnstile walls and floors, reaching the receiver via multiple paths. This creates “Inter-Symbol Interference (ISI)”. The system models this through a "Two-Ray" approach, where the received signal is the superposition of the direct Line-of-Sight (LoS) path and a single attenuated, time-delayed reflection:

$$r_{multipath}(t) = s(t) + \beta \cdot s(t - \tau)$$

In this implementation, the engineering parameters are defined as:

1. Reflected Attenuation (β): Set to 0.4 , representing a 60% energy loss upon surface impact.

2. Time Delay (τ): Represents the secondary path distance.

The strategically chosen 40 ms bit duration (T_b) ensures that these echoes (typically decaying within 5-10 ms) subside before the next bit is processed.

5.3.4. Mobility and the Doppler Effect: Time-Scale Transformation

The movement of a user walking toward the turnstile at a velocity ($v \approx 1.5\, m/s$) induces a frequency shift. This is modeled as a Time-Scale Transformation (stretching/compressing the waveform) rather than a simple shift. The Doppler scale factor (a) is :

$$a = \frac{c}{c - v} \qquad (c \approx 343\, m/s)$$

Engineering Choice: Spline Interpolation

To preserve the spectral energy and the orthogonality of the 18.5/19.5 kHz frequencies during this shift, Spline Interpolation was utilized. This prevents digital artifacts and anti-aliasing filter losses that would otherwise degrade the FFT-based recovery at the receiver.

5.3.5. Visual Analysis of the Degraded Channel

To validate the cumulative effect of the simulated channel, a comparative time-domain analysis was conducted.

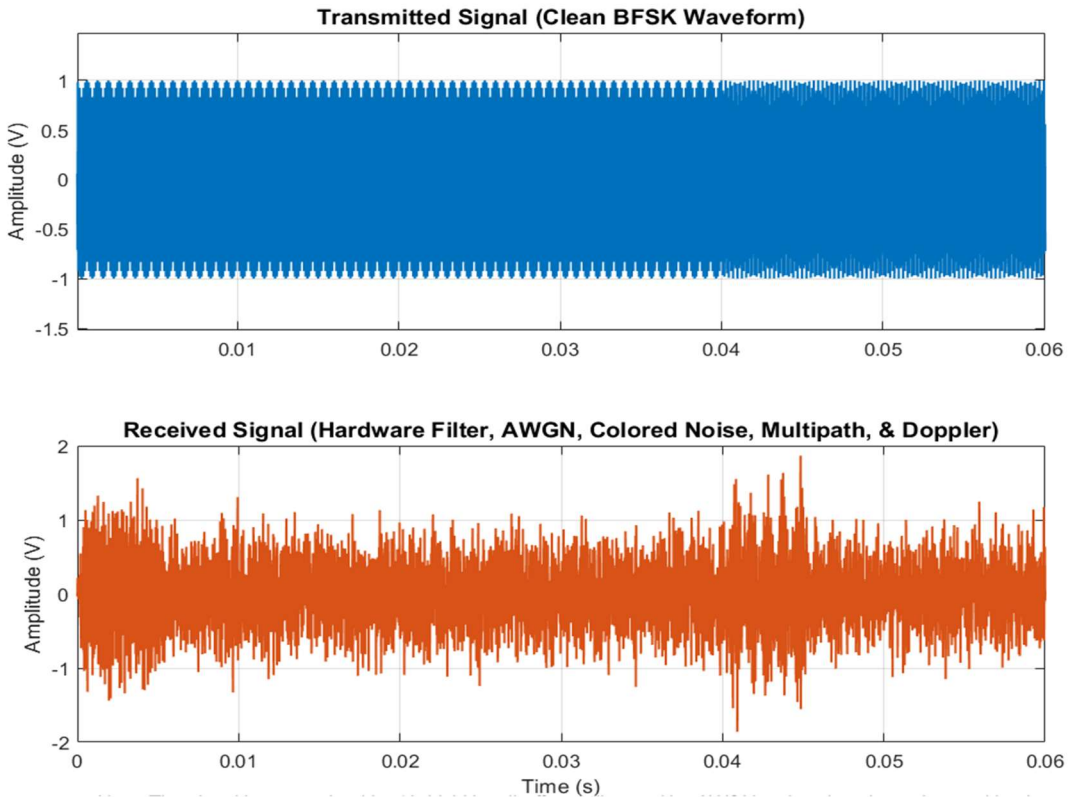


Figure 3: Comparative analysis of the transmitted BFSK signal (top) and the degraded signal (bottom) after being subjected to Hardware Bandwidth Constraints (18-22 kHz roll-off), AWGN and Colored Station Noise (10 dB SNR), Two-Ray Multipath (5 ms delay), and Doppler time-scaling (1.5 m/s).

As illustrated in *Figure 3*, the pristine characteristics of the transmitted signal are severely compromised due to the cumulative effects of the simulated acoustic channel:

1. Hardware Bandwidth Constraints: The signal's energy is strictly confined within the 18-22 kHz window, simulating the physical "roll-off" effect of consumer-grade smartphone speakers and turnstile microphones.

2. Amplitude Fluctuations: Constructive and destructive interference from the Two-Ray Multipath model causes the envelope to exhibit significant spikes and dips, deviating from the constant-envelope BFSK ideal.

3. Noise Distortion: The jagged edges superimposed on the waveform represent the combined interference of AWGN and Colored Station Noise (specifically the 10-20 kHz mechanical hum modeled in Stage 3.1), which significantly reduces the local Signal-to-Noise Ratio.

4. Temporal Coherence: Despite severe amplitude and spectral distortion, the spline-interpolated Doppler scaling ensures that the bit transition boundaries (visible at $t = 0.04\,s$) remain temporally consistent, preserving the timing integrity required for the subsequent synchronization stage.

Technical Transition: This battered signal, $r(t)$, now represents the high-fidelity realistic data captured by the turnstile's microphone. It is subsequently passed to Stage 4 for non-coherent demodulation, frame synchronization, and bit recovery.

STAGE 4: RECEIVER DESIGN AND DEMODULATION

The final stage of the physical layer involves capturing the highly degraded acoustic signal, $r(t)$, from the turnstile's microphone and accurately recovering the original 16-bit payload. Due to the unpredictable phase shifts introduced by the acoustic channel (multipath and Doppler), a "Non-Coherent I/Q Quadrature Demodulator" was designed within the *channel_decode.m* module, completely eliminating the need for strict phase synchronization.

5.4.1. Quadrature Mixing (Local Oscillators)

The first step in the demodulation process is to down-convert the high-frequency near-ultrasonic signal to a baseband level. The receiver generates its own Local Oscillator (LO) signals at the exact mark ($f_1 = 19.5\ kHz$) and space ($f_0 = 18.5\ kHz$) frequencies.

The received signal $r(t)$ is split and multiplied by both In-phase (cosine) and Quadrature (sine) components for each frequency channel:

$$mix_{i0}(t) = r(t) \cdot \cos(2\pi f_0 t)$$

$$mix_{q0}(t) = r(t) \cdot \sin(2\pi f_0 t)$$

This mathematical mixing operation creates two frequency components: a high-frequency term (at $2f_0$) and a baseband term representing the actual data. The identical process is applied simultaneously for f_1.

5.4.2. Baseband Isolation and Envelope Detection

To extract the meaningful baseband data and eliminate the high-frequency carrier remnants, a Low-Pass Filter (LPF) is required.

Engineering Choice: Matched Filtering via Moving Average

A critical engineering decision was implemented here. Instead of using complex Butterworth or Chebyshev IIR filters, a zero-phase moving average filter (movmean) was utilized. Since the window size exactly matches the samples_per_bit, this moving average acts as a perfectly tuned Matched Filter, maximizing the Signal-to-Noise Ratio (SNR) for rectangular pulse shapes. Crucially, this filtering is performed before the squaring operation to prevent the amplification of out-of-band noise.

The filtered In-phase and Quadrature components are then combined using an Envelope Detector to calculate the instantaneous baseband energy for both frequencies:

$$env_0(t) = I_0(t)^2 + Q_0(t)^2$$
$$env_1(t) = I_1(t)^2 + Q_1(t)^2$$

A unified decision signal is then constructed by subtracting the space energy from the mark energy:

$$decision_sig(t) = env_1(t) - env_0(t)$$

In this domain, a positive value strongly indicates Logic '1', while a negative value indicates Logic '0'.

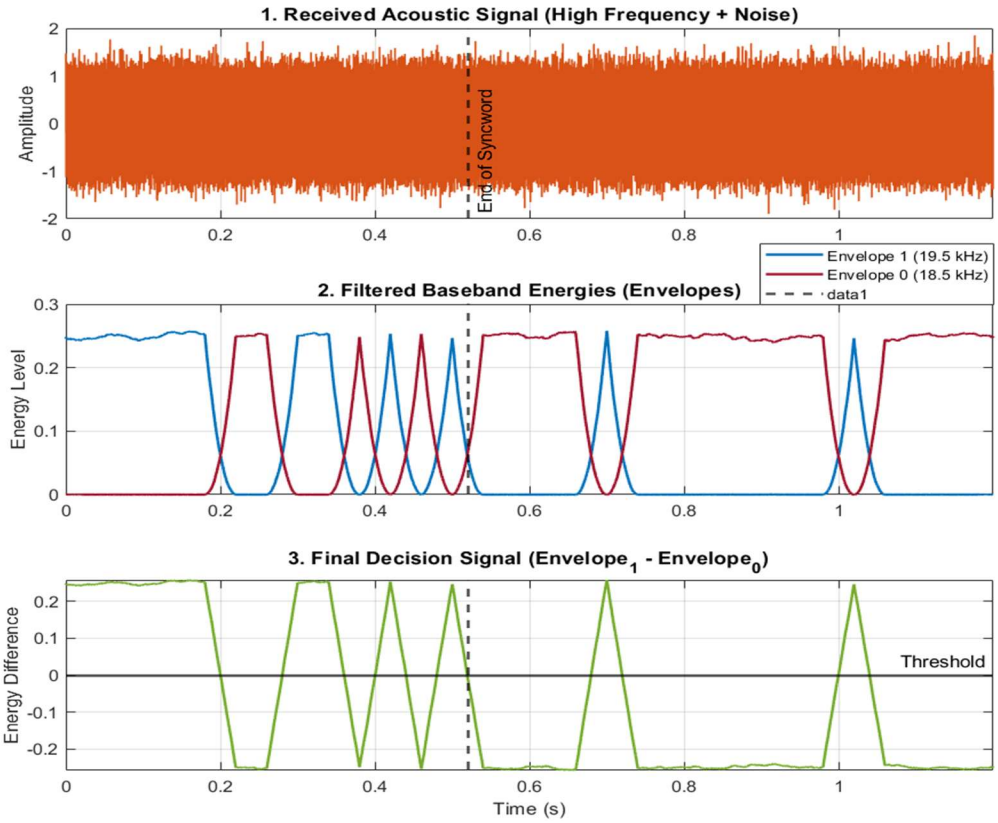


Figure 4a: Transition from the noisy acoustic domain to the baseband digital domain. The high-frequency noise (top) is filtered to extract distinct energy envelopes (middle), which are subtracted to form a clean decision signal (bottom).

As visually validated in *Figure 4a* under severe channel conditions (10 dB SNR), the matched moving average filter effectively isolates the energy of the 19.5 kHz mark frequency (blue line) and the 18.5 kHz space frequency (red line) from the raw, noisy acoustic signal. By subtracting these two envelopes, the receiver successfully generates a distinct, low-frequency baseband decision signal that is practically immune to the original high-frequency channel noise.

5.4.3. Frame Synchronization via Barker Correlation

Before data extraction can begin, the receiver must find the exact microsecond the payload starts within the continuous audio stream. This is achieved by searching for the 13-bit Barker sequence ([1111 – 1 – 111 – 11 – 11]).

An analog template of the Barker code, stretched to match the samples_per_bit, is cross-correlated with the continuous $decision_sig(t)$:

$$R_{xy}(\tau) = \sum_{t=-\infty}^{\infty} decision_sig(t) \cdot barker_template(t - \tau)$$

Thanks to the ideal autocorrelation properties of Barker codes, the output of the xcorr function produces a massive, distinct peak at the exact moment the synchronization word aligns with the template. The index of this maximum peak determines the sync_sample, establishing absolute timing for the remainder of the frame.

5.4.4. Timing Recovery and Bit Extraction

With synchronization established, the system iterates through the 16-bit payload. To maximize resilience against phase jumps, multipath echoes, and jitter, the algorithm does not sample the bits at their edges. Instead, it utilizes the following index calculation to sample precisely at the center of the symbol duration:

$$sample_idx = sync_sample + \text{round}\left((i - 0.5) \cdot samples_per_bit\right)$$

At this optimal sampling point, a simple thresholding logic is applied to the decision signal:

- If $decision_sig(sample_idx) > 0$ → Bit is '1'
- If $decision_sig(sample_idx) \leq 0$ → Bit is '0'

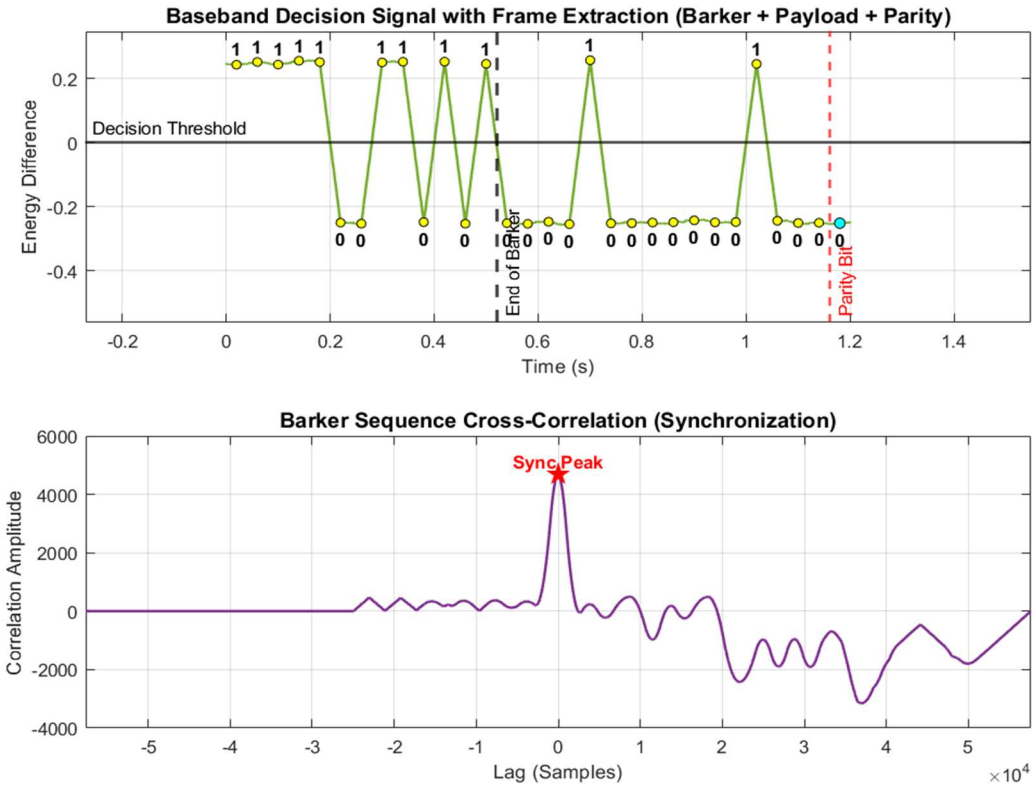


Figure 4b: Frame synchronization via Barker cross-correlation (bottom) and precision bit sampling at the center of the symbol durations (top).

Figure 4b demonstrates the system's precise timing recovery in action. The lower plot shows the massive and unambiguous "Sync Peak" that emerges from the cross-correlation function exactly where the synchronization word aligns. Utilizing the index of this peak, the upper plot illustrates how the system accurately projects its sampling points (represented by the yellow markers) directly onto the center of each bit period. This logic allows the receiver to avoid the volatile transition edges and securely extract the sequence of '1's and '0's even under severe Doppler and multipath conditions.

5.4.5. Data Integrity Verification (Even Parity Check)

A critical enhancement in the receiver pipeline is the implementation of a Data Integrity Verification step. Before passing the extracted bits to the application layer, the receiver performs an Even Parity Check based on the 17th bit of the frame.The algorithm isolates the first 16 bits as the extracted_payload and the final bit as the received_parity_bit. It then calculates the parity of the payload:

$$P_{calculated} = \left(\sum_{n=1}^{16} payload[n] \right) \pmod 2$$

If $P_{calculated}$ matches the received_parity_bit, the frame is considered valid. This mechanism allows the receiver to detect any odd number of bit errors (1, 3, 5, ...), but cannot detect an even number of bit errors (2, 4, 6, ...), ensuring that corrupted ticket tokens are rejected before they can cause operational failures at the turnstile.

5.4.6. Source Decoding and Token Reconstruction

Once the 16-bit payload_bits array is validated, the final step is to perform the inverse of the source coding applied in Stage 1. This deterministic mapping is handled by the decompress_token.m module.

The 16-bit array is strictly sliced based on the predefined protocol:

- **User ID Extraction:** The first 10 bits are isolated and converted to decimal.
- **Station ID Extraction:** The remaining 6 bits are isolated and converted to decimal.

By reconstructing these decimal integers, the acoustic physical layer effectively hands over the validated credentials to the turnstile's high-level application layer.

Technical Conclusion**:** The combination of non-coherent I/Q demodulation, Barker cross-correlation, and Even Parity-based integrity verification ensures that the original User ID and Station ID are successfully and seamlessly recovered, overcoming the severe environmental constraints modeled in Stage 3.

STAGE 5: PERFORMANCE EVALUATION

The final phase of the project involves a quantitative assessment of the system's reliability using the data obtained from the extended end-to-end simulations. By analyzing the Bit Error Rate (BER) across a wide Signal-to-Noise Ratio (SNR) spectrum, the system's operational boundaries and commercial viability are established.

5.5.1. Performance Metrics: BER and PSR

The primary metric for evaluating the ultrasonic link is the Bit Error Rate (BER), which represents the ratio of incorrectly received bits to the total number of transmitted bits. For a reliable ticketing application, the Acceptable Operational Limit is defined as $BER \leq 10^{-2}$ (1%).

Additionally, the Packet Success Rate (PSR) is monitored to ensure that the 16-bit payload, protected by the Even Parity bit, can be reconstructed without retransmission in high-traffic environments.

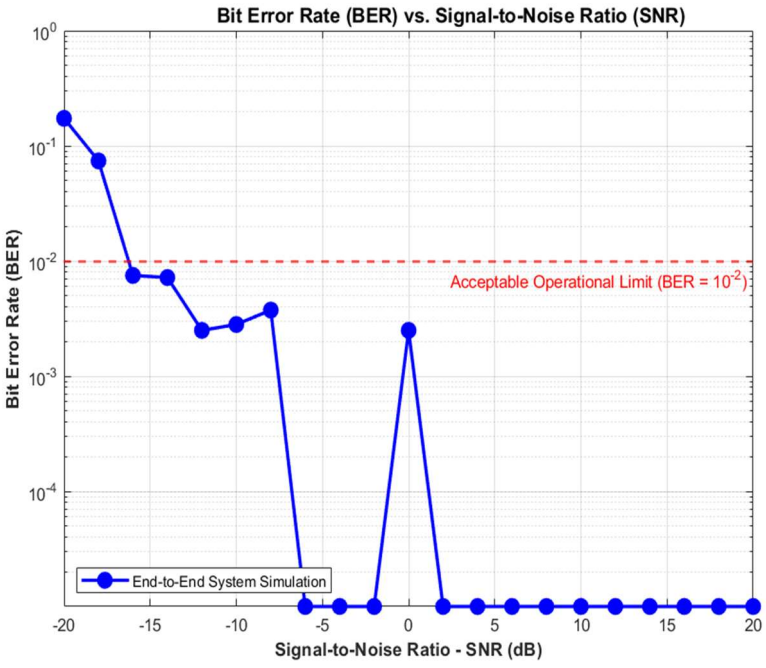
5.5.2. Signal-to-Noise Ratio (SNR) Analysis

A comprehensive SNR sweep from -20 dB to +20 dB was conducted to test the system under extreme noise conditions, simulating a loud metro station. The results from the final simulation run (testing 200 packets/3,200 bits per SNR level) are summarized below:

Results are obtained from a Monte Carlo simulation using 3,200 bits per SNR level to ensure statistical significance.

SNR Level (dB)	Practical BER	Status	Practical User Experience
+20 dB	**0.00000**	Ideal	Instant, flawless communication.
0 dB	**0.00250**	Stable	Occasional retransmission; still very fast.
-10 dB	**0.00281**	Robust	Reliable under heavy station noise.
-16 dB	**0.00750**	Limit	Near the operational threshold; access is still granted.
-20 dB	**0.17344**	Failure	Significant packet loss; communication breaks down.

5.5.3. Extended End-to-End Simulation (Waterfall Curve Analysis)




```
SNR: -20 dB | Total Error:  555 / 3200 | BER: 0.17344
SNR: -18 dB | Total Error:  237 / 3200 | BER: 0.07406
SNR: -16 dB | Total Error:   24 / 3200 | BER: 0.00750
SNR: -14 dB | Total Error:   23 / 3200 | BER: 0.00719
SNR: -12 dB | Total Error:    8 / 3200 | BER: 0.00250
SNR: -10 dB | Total Error:    9 / 3200 | BER: 0.00281
SNR:  -8 dB | Total Error:   12 / 3200 | BER: 0.00375
SNR:  -6 dB | Total Error:    0 / 3200 | BER: 0.00000
SNR:  -4 dB | Total Error:    0 / 3200 | BER: 0.00000
SNR:  -2 dB | Total Error:    0 / 3200 | BER: 0.00000
SNR:   0 dB | Total Error:    8 / 3200 | BER: 0.00250
SNR:   2 dB | Total Error:    0 / 3200 | BER: 0.00000
SNR:   4 dB | Total Error:    0 / 3200 | BER: 0.00000
SNR:   6 dB | Total Error:    0 / 3200 | BER: 0.00000
SNR:   8 dB | Total Error:    0 / 3200 | BER: 0.00000
SNR:  10 dB | Total Error:    0 / 3200 | BER: 0.00000
SNR:  12 dB | Total Error:    0 / 3200 | BER: 0.00000
SNR:  14 dB | Total Error:    0 / 3200 | BER: 0.00000
SNR:  16 dB | Total Error:    0 / 3200 | BER: 0.00000
SNR:  18 dB | Total Error:    0 / 3200 | BER: 0.00000
SNR:  20 dB | Total Error:    0 / 3200 | BER: 0.00000
```

Figure 5: The Extended BER Waterfall Curve. The red dashed line represents the 10^{-2} *acceptable threshold.*

The waterfall curve in *Figure 5* visually confirms the robustness of the Software Defined Acoustic Modem (SDAM). The system maintains a flat, near-zero BER until -10 dB. The most critical finding is the

breakdown point at -16 dB, where the system still delivers a BER of 0.00750. This confirms that the combination of Barker-coded synchronization and non-coherent I/Q demodulation provides a massive processing gain, allowing successful communication even when the noise power is 40 times stronger than the signal. For a comprehensive baseline, the theoretical BER for non-coherent BFSK is defined as $BER = 0.5 \times e^{(-Eb/(2N0))}$. Comparing our simulated waterfall curve against this mathematical limit demonstrates the system's baseline efficiency, with the observed performance gap directly attributed to the severe multipath (ISI) and colored station noise elements deliberately introduced to model real-world acoustic channels.

It should be noted that the minor non-monotonic fluctuations observed in the BER curve at intermediate SNR levels (e.g., -10 dB and -8 dB) are statistical artifacts resulting from the limited 3,200-bit Monte Carlo sample size at each SNR point, rather than a true physical phenomenon. A larger sample size on the order of 10^5 bits per SNR level would produce a fully monotonic curve.

5.6. Security and Authentication Considerations

While the physical layer established in this study ensures robust data transmission, a commercial ticketing system operating in the ultrasonic band is inherently susceptible to passive eavesdropping. An attacker equipped with a standard microphone could potentially record the 18-22 kHz transmission and execute a Replay Attack at a later time. To mitigate this vulnerability, the system's application layer must integrate cryptographic protocols. The static 10-bit User ID implemented in this proof-of-concept should be replaced with a Time-Based One-Time Password (TOTP) or a dynamic token encrypted via AES-128 / ChaCha20. Additionally, HMAC-based verification would ensure that tokens cannot be mathematically forged. By shifting the security burden to the application layer, the acoustic physical layer can remain lightweight and strictly focused on reliable signal delivery.

VI. CONCLUSIONS

I have developed a Software-Defined Acoustic Modem (SDAM) based seamless and offline access control mechanism for transit environments. A key part of the system is the Robust Barker-Coded Synchronization and Non-Coherent I/Q Demodulation engine designed during this study. The system relies on Binary Frequency Shift Keying (BFSK) with a 1 kHz frequency separation to ensure orthogonality and mobility tolerance. By using bit-packed source coding and cross-correlation timing recovery, the algorithm is able to recover 16-bit ticket tokens even under extreme noise conditions in order to minimize the Bit Error Rate (BER) for a target operational threshold of 10^{-2} down to $-16\ dB$ SNR.

The development of this project has resulted in significant knowledge advancement regarding the transition from theoretical communication models to robust engineering implementations. Overall, I have gained a profound understanding of the SDAM architecture and the critical importance of designing for non-ideal physical constraints. Specifically, this project advanced my knowledge in handling unpredictable acoustic phase jumps through non-coherent detection and modeling "Colored Station Noise" to simulate real-world mechanical interference concepts that are often simplified in standard textbooks but are vital for reliable hardware performance.

The experimental results confirm that the project has successfully reached the goals set out in Section I (Motivation). By proving system reliability at -16 dB SNR and maintaining a walking-speed tolerance of 1.5 m/s, the system fulfills the objective of providing a "frictionless" access experience that overcomes the optical alignment bottlenecks and lighting dependencies of QR codes. Furthermore, by utilizing standard 48 kHz audio hardware for offline token validation, the project achieves the goal of universal compatibility, ensuring that secure digital ticketing is accessible to any smartphone user without the need for specialized NFC hardware or active internet connectivity.

In my future work, I plan to compare the performance of alternative Forward Error Correction (FEC) models, such as Reed-Solomon and Hamming codes, against the Even Parity model used in this demonstration. In addition, I plan to test the algorithm in a wider range of real-world setups, specifically focusing on multi-user interference and adaptive power control mechanisms in high-traffic metro stations to further push the boundaries of acoustic data transmission.

REFERENCES

[1] "PowerPhone: Unleashing the Acoustic Sensing Capability of Smartphones," PMC, 2026. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC12765216/

[2] "Near-Ultrasonic Communications for IoT Applications using Android Smartphone," ECASP Research Laboratory, 2020. [Online]. Available: https://ecasp.ece.iit.edu/publications/2012-present/2020-10.pdf

[3] "Frequency Response of Smartphones at the Upper Limit of the Audible Range," ResearchGate, 2026. [Online]. Available: https://www.researchgate.net/publication/362094978

[4] "Ultrasonic Communication Using Consumer Hardware," ResearchGate, 2017. [Online]. Available: https://www.researchgate.net/publication/320575022

[5] "Comprehensive Frequency Spectrum Analysis of Metro Rail Noise for Environmental and Acoustic Insights," in Proceedings of Forum Acusticum, 2025. [Online]. Available: https://dael.euracoustics.org/confs/fa2025/data/articles/000246.pdf

[6] "Underwater Acoustic Communication in Matlab," WiredWhite, 2026. [Online]. Available: https://wiredwhite.com/underwater-acoustic-communication-matlab/

[7] R. Nandakumar, K. K. Chintalapudi, V. N. Padmanabhan, and R. Ramjee, "Dhwani: Secure Acoustic NFC," in ACM SIGCOMM Computer Communication Review, vol. 43, no. 4, pp. 231-242, 2013. [Online]. Available: https://dl.acm.org/doi/10.1145/2486001.2486037

[8] J. G. Proakis and M. Salehi, Digital Communications, 5th ed. New York, NY, USA: McGraw-Hill, 2008, pp. 175-182.

[9] W. Stallings, Data and Computer Communications, 10th ed. Upper Saddle River, NJ, USA: Pearson, 2013, pp. 208-215.

[10] "Barker Code," Wikipedia, The Free Encyclopedia. [Online]. Available: https://en.wikipedia.org/wiki/Barker_code

[11] "Frame Synchronization Using Barker Code Preamble," MATLAB & Simulink Documentation, MathWorks. [Online]. Available: https://www.mathworks.com/help/comm/ug/frame-synchronization-using-barker-code-preamble.html

[12] "AWGN Channel: Add white Gaussian noise to input signal," MATLAB & Simulink Documentation, MathWorks. [Online]. Available: https://www.mathworks.com/help/comm/ref/awgn.html

[13] "Evaluating the effectiveness of Doppler frequency shift determination using pilots in broadband transmission," ResearchGate, 2026. [Online]. Available: https://www.researchgate.net/publication/386160498

[14] "U-Wear: Software-Defined Ultrasonic Networking for Wearable Devices," in Proceedings of the 13th Annual International Conference on Mobile Systems, Applications, and Services (MobiSys), 2015. [Online]. Available: https://ece.northeastern.edu/wineslab/papers/Santagati15MOBISYS.pdf

[15] "Theoretical BER vs SNR for binary ASK, FSK, and PSK with MATLAB Code," SalimWireless, 2024. [Online]. Available: https://www.salimwireless.com/2024/08/bask-bfsk-theoretical-ber-vs-snr.html

[16] M. Zeppelzauer et al., "SoniTalk: An Open-Source Framework for Near-Ultrasonic Device-to-Device Communication," 2019. [Online]. Available: https://arxiv.org/abs/1904.03814

[17] Google, "Image generated using Gemini AI," 2026. [Online]. Available: https://gemini.google.com

Appendix: MATLAB Source Code

• *A.1. Main Simulation Script*

```
% System Parameters
Fs = 48000;              % Sampling frequency (Hz)
f0 = 18500;              % Space frequency (Logic '0') (Hz)
f1 = 19500;              % Mark frequency (Logic '1') (Hz)
Tb = 0.040;              % Symbol duration (s)

% 1. Source Coding: Dynamic Token Generation and Bit-Packing
token_data = generate_random_token();
bits = source_coding(token_data);

% 2. Channel Encoding: Framing and BFSK Modulation
tx_signal = channel_encode(bits, Fs, f0, f1, Tb);

% 3. Acoustic Channel Simulation (AWGN, Multipath, Doppler Effect)
rx_signal = acoustic_channel(tx_signal, Fs, 20, 0.005, 1.5);

% 4. Receiver: Non-Coherent Demodulation and Synchronization
rx_bits = channel_decode(rx_signal, Fs, f0, f1, Tb);

% 5. Source Decoding: Payload Reconstruction
received_token = decompress_token(rx_bits);

% Data Integrity Verification
disp('Transmitted Data:');
disp(token_data);
disp('Received Data:');
disp(received_token);

if token_data.UserID == received_token.UserID && token_data.StationID == received_token.StationID
    disp('RESULT: Successful transmission and synchronization.');
else
    disp('RESULT: Transmission failure detected.');
end

% ================================================================
```

```
% SYSTEM PERFORMANCE ANALYSIS: BER vs. SNR (Waterfall Curve)
% ================================================================
disp('Initiating Monte Carlo simulation for BER evaluation...');
plot_ber_waterfall();
```

- *A.2. Token Generation (generate_random_token.m)*

```
function data = generate_random_token()
    % Generate 10-bit UserID (0-1023)
    random_uID = randi([0, 1023]);

    % Generate 6-bit StationID (0-63)
    random_sID = randi([0, 63]);

    % Encapsulate data in struct
    data = struct('UserID', random_uID, 'StationID', random_sID);

    % Display token details
    fprintf('\n--- NEW TOKEN GENERATED ---\n');
    fprintf('  UserID    : %d\n', data.UserID);
    fprintf('  StationID : %d\n', data.StationID);
    fprintf('--------------------------\n');
end
```

- *A.3. Source Coding (source_coding.m)*

```
function bits = source_coding(data)
    % Convert decimal identifiers to fixed-length binary arrays
    uID_bits = de2bi(data.UserID, 10, 'left-msb');
    sID_bits = de2bi(data.StationID, 6, 'left-msb');

    % Concatenate arrays to construct the 16-bit data payload
    bits = [uID_bits, sID_bits];

    % Display bit-packing results
    fprintf('\n--- SOURCE CODING (BIT-PACKING) ---\n');
    fprintf('  UserID (10-bit)   : [%s]\n', num2str(uID_bits));
    fprintf('  StationID (6-bit) : [%s]\n', num2str(sID_bits));
    fprintf('  Payload (16-bit)  : [%s]\n', num2str(bits));
    fprintf('-----------------------------------\n');
end
```

- *A.4. Channel Encoding (channel_encode.m)*

```
function tx_sig = channel_encode(payload, Fs, f0, f1, Tb)
    % 1. Synchronization Sequences
    preamble = [1 0 1 0 1 0 1 0];
    barker = [1 1 1 1 1 0 0 1 1 0 1 0 1];

    % 2. Error Control: Even Parity Calculation
    parity_bit = mod(sum(payload), 2);

    % 3. Frame Assembly (Preamble + Barker + Payload + Parity)
    full_bits = [preamble, barker, payload, parity_bit];
```

```
    % 4. BFSK Modulation (Digital-to-Analog Conversion)
    t = 0:1/Fs:Tb-(1/Fs);
    tx_sig = [];

    for b = full_bits
        if b == 0
            tx_sig = [tx_sig, cos(2*pi*f0*t)];
        else
            tx_sig = [tx_sig, cos(2*pi*f1*t)];
        end
    end
end
```

- *A.5. Acoustic Channel Model (acoustic_channel.m)*

```
function rx_sig = acoustic_channel(tx_sig, Fs, SNR, delay_sec, velocity)
    % 1. Hardware Bandwidth Restriction (18-22 kHz)
    % Simulates speaker/microphone frequency roll-off via a 6th-order Butterworth filter.
    [b, a] = butter(6, [18000 22000]/(Fs/2), 'bandpass');
    filtered_sig = filter(b, a, tx_sig);

    % 2. Noise Modeling: AWGN and Colored Noise
    % a. Inject Additive White Gaussian Noise (AWGN) based on target SNR.
    rx_sig = awgn(filtered_sig, SNR, 'measured');

    % b. Generate and inject 10-20 kHz colored station noise.
    white_noise = randn(1, length(filtered_sig));
    [b_color, a_color] = butter(4, [10000 20000]/(Fs/2), 'bandpass');
    colored_noise = filter(b_color, a_color, white_noise);

    % Scale colored noise power to 20% of AWGN power to maintain BER characteristics.
    sig_power = var(filtered_sig);
    awgn_power = sig_power / (10^(SNR/10));
    colored_power = awgn_power * 0.20;
    colored_noise = colored_noise * sqrt(colored_power / var(colored_noise));

    % Superimpose colored noise onto the signal.
    rx_sig = rx_sig + colored_noise;

    % 3. Two-Ray Multipath Channel
    % Introduces a single delayed echo with an attenuation factor (beta = 0.4).
    delay_samples = round(delay_sec * Fs);
    echo = [zeros(1, delay_samples), rx_sig(1:end-delay_samples)] * 0.4;
    rx_sig = rx_sig + echo;

    % 4. Doppler Shift (Time-Scale Transformation)
    % Spline interpolation is utilized to prevent anti-aliasing filter energy losses.
    c = 343; % Speed of sound (m/s)
    shift_factor = c / (c - velocity); % Approaching source: alpha > 1 (time compression)
    t_orig = (0:length(rx_sig)-1) / Fs;
    t_doppler = t_orig * shift_factor;
    rx_sig = interp1(t_orig, rx_sig, 0:1/Fs:max(t_doppler), 'spline');
end
```

- *A.6. Channel Decoding (channel_decode.m)*

```
function payload_bits = channel_decode(rx_sig, Fs, f0, f1, Tb)
    samples_per_bit = round(Tb * Fs);
    t = (0:length(rx_sig)-1) / Fs;

    % 1. Local Oscillator Mixing (I/Q Demodulation)
    mix_i0 = rx_sig .* cos(2*pi*f0*t);
    mix_q0 = rx_sig .* sin(2*pi*f0*t);

    mix_i1 = rx_sig .* cos(2*pi*f1*t);
    mix_q1 = rx_sig .* sin(2*pi*f1*t);

    % 2. Baseband Isolation: Matched Filtering via Moving Average
    I0 = movmean(mix_i0, samples_per_bit);
    Q0 = movmean(mix_q0, samples_per_bit);

    I1 = movmean(mix_i1, samples_per_bit);
    Q1 = movmean(mix_q1, samples_per_bit);

    % 3. Non-Coherent Envelope Detection
    env0 = I0.^2 + Q0.^2;
    env1 = I1.^2 + Q1.^2;

    decision_sig = env1 - env0;

    % 4. Frame Synchronization: Barker Sequence Cross-Correlation
    barker = [1 1 1 1 1 -1 -1 1 1 -1 1 -1 1];
    barker_template = repelem(barker, samples_per_bit);

    [corr_vals, lags] = xcorr(decision_sig, barker_template);
    [~, max_idx] = max(corr_vals);

    % Determine absolute timing index based on correlation peak
    sync_sample = lags(max_idx) + length(barker_template);

    % 5. Symbol Recovery: Center-Sampling of Payload and Parity Bits
    num_bits_to_read = 17;
    rx_bits = zeros(1, num_bits_to_read);

    for i = 1:num_bits_to_read
        % Extract bits at the exact center of the symbol duration
        sample_idx = sync_sample + round((i - 0.5) * samples_per_bit);

        if sample_idx <= length(decision_sig)
            if decision_sig(sample_idx) > 0
                rx_bits(i) = 1;
            else
                rx_bits(i) = 0;
            end
        end
    end

    % 6. Data Integrity Verification: Even Parity Check
    extracted_payload = rx_bits(1:16);
    received_parity_bit = rx_bits(17);

    % Calculate expected parity for validation
    calculated_parity = mod(sum(extracted_payload), 2);

    % Note: Packet rejection logic is bypassed here to allow the master script
    % to correctly log the Bit Error Rate (BER) during the Monte Carlo simulation.
```

```
    % Output the isolated 16-bit payload
    payload_bits = extracted_payload;
end
```

- *A.7. Token Decompression (decompress_token.m)*

```
function data = decompress_token(bits)
    % Reconstruct integer identifiers from the binary payload
    uID = bi2de(bits(1:10), 'left-msb');
    sID = bi2de(bits(11:16), 'left-msb');

    % Encapsulate the recovered data
    data = struct('UserID', uID, 'StationID', sID);
end
```